

Advanced Certification Programme in Agentic and Generative AI

A Programme by IISc and TalentSprint

Additional Notebook: **Prompt Engineering**

(Ungraded)

Objective

Experience how different prompt engineering techniques can shape the quality, accuracy, and usefulness of AI outputs. The goal is to highlight how thoughtful prompt design enhances reasoning, creativity, and control compared to generic prompting.

Instructions

- First, get familiar with different Prompting Techniques by following the steps given in the notebook.
- Then, complete the ungraded practice task given at the end of the notebook.

I. Getting Started with different Prompting Techniques

Information

Prompt is the input you give to an AI model—a piece of text, instruction, or query that tells the model what you want it to do.

Prompt Engineering is the practice of carefully designing, crafting, and refining the *input* ("prompts") given to generative AI models to achieve desired *outputs*. It involves using clear language, context, and instructions to guide the AI toward producing relevant, accurate, and specific responses, much like providing a roadmap or detailed instructions to someone.

Elements of a Prompt

A prompt contains any of the following elements:

- **Instruction** - a specific task or instruction you want the model to perform
 - **Context** - external information or additional context that can steer the model to better responses
 - **Input Data** - the input or question that we are interested to find a response for
 - **Output Indicator** - the type or format of the output.
-

Groq API

What is Groq?

Groq is a technology company that builds **specialized AI hardware and software** designed to run large language models (LLMs) and other AI models **extremely fast and efficiently**, especially for **real-time inference**.

This notebook demonstrates how to interact with the **OpenAI and Llama models** using Groq API.

It covers the necessary steps from installing the required packages and setting up authentication to making a chat completion request and processing the response. The example prompt is designed to generate a detailed essay on a specific topic.

Read the Groq API Key

Using the Groq API Key, you will have access to the OpenAI and Llama model, free of cost, under the [Free-tier](#).

This cell retrieves the Groq API key securely stored in Google Colab's Secrets (see 'key' icon on the leftmost panel of this notebook) and sets it as an environment variable `GROQ_API_KEY`. This is a recommended practice to avoid exposing your API key directly in the code.

- Go to <https://console.groq.com/>, and setup a Free account.
- Create a new API by visiting: <https://console.groq.com/keys>
- Save the key in Google Colab's Secrets

```
Secret Name: GROQ_API_KEY
Secret Value: Paste your Groq api key
```

- Read the key and save as an environment variable `GROQ_API_KEY`

```
# Save the key in Colab's Secrets then load from there

import os
from google.colab import userdata

os.environ['GROQ_API_KEY'] = userdata.get('GROQ_API_KEY')
```

Create Groq Client

This cell initializes the Groq client using the API key stored in the environment variable. This client object will be used to make API calls to OpenAI model.

```
# Install Groq library
!pip -q install groq
```

142.3/142.3 kB 2.3 MB/s eta 0:00:00

```
from groq import Groq
```

```
# Initialize Groq client
groq_client = Groq()
```

1) Zero-Shot Prompting

Large language models (LLMs) today, such as GPT-3.5 Turbo, GPT-4, and Claude 3, are tuned to follow instructions and are trained on large amounts of data. Large-scale training makes these models capable of performing some tasks in a "zero-shot" manner. Zero-shot prompting means that the prompt used to interact with the model won't contain examples or demonstrations. **The zero-shot prompt directly instructs the model to perform a task without any additional examples to steer it.**

Here is one of example for text classification:

Prompt:

```
Classify the text into neutral, negative or positive.
Text: I think the vacation is okay.
Sentiment:
```

```
prompt = """Classify the text into neutral, negative or positive.
Text: I think the vacation is okay.
Sentiment:
"""
```

```
response = groq_client.chat.completions.create(
    model="openai/gpt-oss-20b",
    messages=[
        {"role": "user", "content": prompt}
    ]
)
```

```
print(response.choices[0].message.content)
```

```
Sentiment: neutral
```

When zero-shot doesn't work, it's recommended to provide demonstrations or examples in the prompt which leads to few-shot prompting.

2) Few-Shot Prompting

While large-language models demonstrate remarkable zero-shot capabilities, they still fall short on more complex tasks when using the zero-shot setting. Few-shot prompting can be used as a technique to enable in-context learning where we provide demonstrations in the prompt to steer the model to better performance. The demonstrations serve as conditioning for subsequent examples where we would like the model to generate a response.

Here is one of example for text classification:

Prompt:

```
This is awesome! // Positive
This is bad! // Negative
Wow that movie was rad! // Positive
What a horrible show! //
```

```
prompt = """This is awesome! // Positive
This is bad! // Negative
Wow that movie was rad! // Positive
What a horrible show! //
"""

response = groq_client.chat.completions.create(
    model="openai/gpt-oss-20b",
    messages=[
        {"role": "user", "content": prompt}
    ]
)

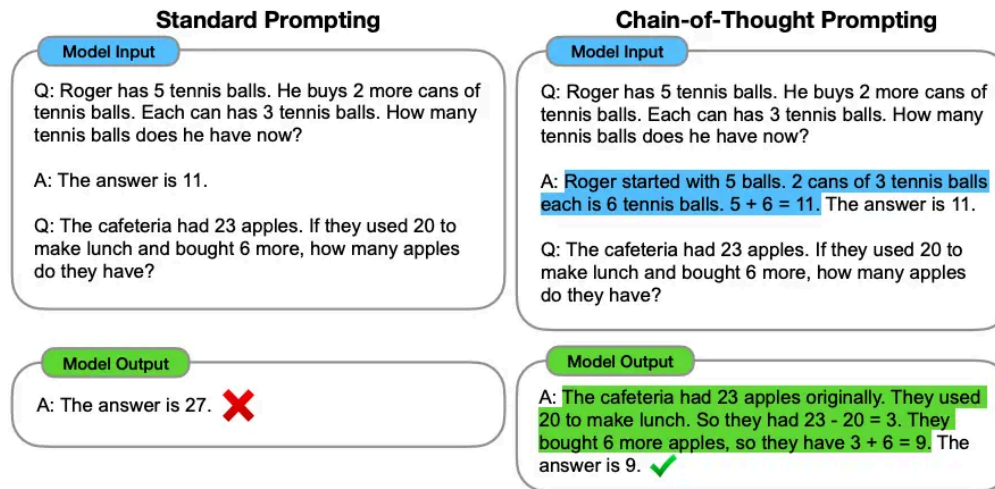
print(response.choices[0].message.content)
```

Negative

Standard few-shot prompting works well for many tasks but is still not a perfect technique, especially when dealing with more complex reasoning tasks.

When zero-shot prompting and few-shot prompting are not sufficient, it might mean that whatever was learned by the model isn't enough to do well at the task. From here it is recommended to start thinking about fine-tuning your models or experimenting with more advanced prompting techniques.

3) Chain-of-Thought (CoT) Prompting



[Image Source](#)

Introduced in [Wei et al. \(2022\)](#), chain-of-thought (CoT) prompting enables complex reasoning capabilities through intermediate reasoning steps. You can combine it with few-shot prompting to get better results on more complex tasks that require reasoning before responding.

For example:

Prompt:

I went to the market and bought 10 apples. I gave 2 apples to the neighbor and 2 to the repairman.
I then went and bought 5 more apples and ate 1. How many apples did I remain with?
Let's think step by step.

```
prompt = """I went to the market and bought 10 apples. I gave 2 apples to the neighbor and 2 to the repairman.
I then went and bought 5 more apples and ate 1. How many apples did I remain with?
Let's think step by step.
"""
```

```
response = groq_client.chat.completions.create(
    model="openai/gpt-oss-20b",
    messages=[
        {"role": "user", "content": prompt}
    ]
)
```

```
print(response.choices[0].message.content)
```

Let's work it out in stages.

Step	Action	Apples you have
1	Bought 10 apples	10
2	Gave 2 to the neighbor	$10 - 2 = 8$
3	Gave 2 to the repairman	$8 - 2 = 6$
4	Bought 5 more apples	$6 + 5 = 11$
5	Ate 1 apple	$11 - 1 = 10$

Answer: You're left with 10 apples.

4) System Prompts (Assigning a Role)

A system prompt is a hidden or background instruction given to an AI model that sets the rules, behavior, or persona of the AI before it responds to user queries.

- It's not usually visible to the end-user.
- It defines how the AI should think, respond, and behave.
- It acts like the "ground rules" or operating instructions for the AI.

Difference from User Prompt

- **System Prompt:** Defines the AI's role, style, and boundaries. (invisible, set by developers or platform)
- **User Prompt:** The actual question or instruction given by the user.

Why It Matters

- Ensures consistent AI behavior across users.
- Prevents harmful, biased, or off-topic outputs.
- Useful for persona-based AI (e.g., tutor, lawyer, doctor assistant).

For Example:

System Prompt:

You are a very smart physics professor.
 You are great at answering questions about physics in a concise and easy to understand manner.
 When you don't know the answer to a question you admit that you don't know.
 Start your answer with `Subject:Physics :`

Prompt:

What is black body radiation?

```
system_prompt = """You are a very smart physics professor. \
You are great at answering questions about physics in a concise and easy to understand manner. \
When you don't know the answer to a question you admit that you don't know. \
Start your answer with `Subject:Physics :` \
"""
```

```
prompt = "What is black body radiation?"
```

```

response = groq_client.chat.completions.create(
    model="openai/gpt-oss-20b",
    messages=[
        {"role": "system", "content": system_prompt},
        {"role": "user", "content": prompt}
    ]
)

print(response.choices[0].message.content)

```

Subject:Physics :

Black-body radiation is the electromagnetic radiation that a perfect absorber (a “black body”) would emit when it is in thermal equilibrium.

- **Perfect absorber:** absorbs all incoming radiation regardless of frequency.
- **Thermal equilibrium:** its temperature is uniform, and its emitted spectrum does not change with time.

Because the body is a perfect emitter as well as a perfect absorber (Kirchhoff’s law), its emitted intensity depends only on its temperature, not on its material or shape.

Key results:

1. **Planck’s law** (1900) gives the spectral radiance $B_{\nu}(T)$ or $B_{\lambda}(T)$ as a function of frequency ν or wavelength λ and temperature T .
2. **Wien’s displacement law** states that the wavelength at which the spectrum peaks shifts inversely with temperature: $\lambda_{\text{max}} \approx \frac{b}{T}$ (with $b \approx 2.9 \times 10^{-3} \text{ m}\cdot\text{K}$).
3. **Stefan-Boltzmann law** gives the total emitted power per unit area: $P = \sigma T^4$, where σ is the Stefan-Boltzmann constant.

Black-body radiation is the theoretical basis for understanding stellar spectra, the cosmic microwave background, and many modern technologies (e.g., incandescent bulbs, thermal cameras).

```
prompt2 = "What is Vibe Coding?"
```

```

response = groq_client.chat.completions.create(
    model="openai/gpt-oss-20b",
    messages=[
        {"role": "system", "content": system_prompt},
        {"role": "user", "content": prompt2}
    ]
)

print(response.choices[0].message.content)

```

Subject:Physics :

Vibe Coding is not a physics term, but it’s a phrase that shows up in a few different tech circles.

The most common references are:

Context	What it means
Vibe.d web framework	“Vibe Coding” can simply mean writing code for the lightweight, async web framework Vibe.d (a D-language web framework). Developers often refer to themselves as Vibe Coders .
Coding style / vibe	Outside of Vibe.d, some communities use the phrase to describe a <i>coding vibe</i> —a style that values readability, maintainability, and a positive, collaborative development culture.
Educational or community projects	A handful of bootcamps or online communities (e.g., “Vibe Coding Academy”) use the name to brand their curriculum, though these are not widely standardized.

If you’re looking for a specific framework, it’s most likely Vibe.d. If you meant a broader “coding vibe” approach, it’s an informal way to describe clean, well-structured, and enjoyable code.

5) Prompt Chaining

To improve the reliability and performance of LLMs, one of the important prompt engineering techniques is to **break tasks into its subtasks**. Once those subtasks have been identified, the LLM is prompted with a subtask and then its **response is used as input to**

another prompt. This is what's referred to as prompt chaining, where a task is split into subtasks with the idea to create a chain of prompt operations.

Prompt chaining is useful to accomplish complex tasks which an LLM might struggle to address if prompted with a very detailed prompt. In prompt chaining, chain prompts perform transformations or additional processes on the generated responses before reaching a final desired state.

Besides achieving better performance, prompt chaining helps to boost the transparency of your LLM application, increases controllability, and reliability. This means that you can debug problems with model responses much more easily and analyze and improve performance in the different stages that need improvement.

Prompt chaining is particularly useful when building LLM-powered conversational assistants and improving the personalization and user experience of your applications.

For Example: Let us consider an example to see how a large task—writing a blog post on remote work—can be broken into smaller steps using Prompt Chaining.

- First, the AI generates a blog outline,
- then expands one section into detailed text,
- summarizes that section into concise bullet points, and
- finally creates a persuasive call-to-action.

This step-by-step approach ensures clarity, structure, and refined outputs instead of trying to get everything in a single prompt.

```
# Step 1 - Generate an Outline
```

```
outline_prompt = "Create a 5-point outline for a blog post on the benefits of remote work."
```

```
outline = groq_client.chat.completions.create(  
    model="openai/gpt-oss-20b",  
    messages=[  
        {"role": "user", "content": outline_prompt}  
    ]  
)  
outline = outline.choices[0].message.content  
  
print(outline)
```

```
**5-Point Outline for a Blog Post: "The Benefits of Remote Work"**
```

1. **Introduction**
 - Brief overview of the rise of remote work
 - Thesis statement: Remote work offers tangible benefits for employees, employers, and the broader economy
2. **Flexibility & Autonomy**
 - **Work-time freedom:** Ability to set personal schedules that align with peak productivity
 - **Location independence:** Work from anywhere—home, café, or a co-working space
 - **Reduced commuting stress:** More time for personal activities and rest
3. **Increased Productivity & Focus**
 - **Customizable work environments:** Employees can create a space that maximizes concentration
 - **Fewer office interruptions:** Reduced meetings and office chatter lead to deeper work
 - **Data on output:** Statistics and case studies showing higher output per hour
4. **Cost Savings for All Stakeholders**

- ***For employees***: Lower commuting costs, reduced need for professional wardrobe, cheaper meals
- ***For employers***: Savings on office space, utilities, and facilities maintenance
- ***Broader economic impact***: Lower traffic congestion and environmental benefits

5. ****Talent Acquisition & Retention****

- ***Global talent pool***: Ability to hire the best fit regardless of geography
- ***Increased employee satisfaction***: Remote work as a key retention lever
- ***Diversity & inclusion***: Broader representation through flexible work options

6. ****Conclusion & Call to Action****

- Recap of key benefits
- Encourage businesses to adopt hybrid or fully remote policies
- Invite readers to share their remote work experiences or ask questions

(Feel free to expand each section with anecdotes, data, or practical tips to make the post engaging and actionable.)

Step 2 - Expand a Section

expand_prompt = "Expand point #2 into a 120-word explanation with real-life examples."

```
point2_explanation = groq_client.chat.completions.create(
    model="openai/gpt-oss-20b",
    messages=[
        {"role": "user", "content": outline_prompt},
        {"role": "assistant", "content": outline},
        {"role": "user", "content": expand_prompt},
    ]
).choices[0].message.content

print(point2_explanation)
```

****Flexibility & Autonomy****

Flexibility & autonomy allow employees to shape their schedules around peak creativity, family needs, or personal health. For instance, David, a senior developer at a fintech startup, choo

Step 3 - Create a Summary

summary_prompt = "Summarize this section in 2 bullet points for a presentation slide."

```
point2_summary = groq_client.chat.completions.create(
    model="openai/gpt-oss-20b",
    messages=[
        {"role": "user", "content": outline_prompt},
        {"role": "assistant", "content": outline},
        {"role": "user", "content": expand_prompt},
        {"role": "assistant", "content": point2_explanation},
        {"role": "user", "content": summary_prompt},
    ]
).choices[0].message.content

print(point2_summary)
```

- ****Work-time freedom****: Employees schedule hours around peak productivity and personal needs, cutting commute time and boosting morale.
- ****Autonomy drives retention****: Flexible, location-independent arrangements lead to higher job satisfaction and lower absenteeism.

```
# Step 4 - Add a Call-to-Action

call_to_action_prompt = "Write a closing call-to-action encouraging companies to adopt remote work policies."

call_to_action = groq_client.chat.completions.create(
    model="openai/gpt-oss-20b",
    messages=[
        {"role": "user", "content": outline_prompt},
        {"role": "assistant", "content": outline},
        {"role": "user", "content": expand_prompt},
        {"role": "assistant", "content": point2_explanation},
        {"role": "user", "content": summary_prompt},
        {"role": "assistant", "content": point2_summary},
        {"role": "user", "content": call_to_action_prompt},
    ]
).choices[0].message.content

print(call_to_action)
```

****Ready to transform your workforce?****
 Start today: pilot a flexible remote-work plan for one department, gather data, and scale what works.
 Give your talent the freedom it deserves, unlock higher productivity, and position your company at the forefront of tomorrow's workplace.
 Let's build a future where work adapts to people, not the other way around. 🚀

6) Using an LLM to provide Prompt

Instead of framing the prompt ourself, we can ask an LLM to create a detailed prompt that can be used to get the desired response.

Example Scenario: Suppose we want to generate a professional email, but instead of manually writing the prompt, we first ask the LLM to create a better prompt for us.

```
# Function to generate prompt

def generate_prompt(input):
    prompt = f"""For the given request create a detailed prompt to be fed to an LLM for response generation.
    Provide only the prompt and not the LLM response.
    Request: {input}"""

    response = groq_client.chat.completions.create(
        model="openai/gpt-oss-120b",
        messages=[
            {"role": "user", "content": prompt}
        ]
    )

    return response.choices[0].message.content
```

```
request = "write a professional email requesting a meeting with a client next week"
new_prompt = generate_prompt(request)

print(new_prompt)
```

****Prompt for the LLM****

You are an experienced business communication specialist. Write a concise, courteous, and professional email to a client requesting a meeting next week. Follow these guidelines:

1. ****Recipient Details****

- Use a placeholder for the client's name (e.g., ****[Client Name]****).
- Use a placeholder for the client's company (e.g., ****[Client Company]****).

2. ****Sender Details****

- Include placeholders for your name (****[Your Name]****), title (****[Your Title]****), and organization (****[Your Company]****).
- Provide a professional email signature block (name, title, company, phone, and email).

3. ****Subject Line****

- Craft a clear, specific subject line that indicates the purpose (e.g., "Request for Meeting Next Week - [Project/Topic]").

4. ****Opening****

- Begin with a polite greeting (e.g., "Dear [Client Name],").
- Reference any recent interaction or context that justifies the meeting (e.g., "Following our recent conversation about...").

5. ****Purpose of the Email****

- State clearly that you would like to schedule a meeting next week to discuss a specific topic or project (****[Topic/Project]****).

6. ****Proposed Dates/Times****

- Offer 2-3 specific date and time options (including time zone) that you are available.
- Invite the client to suggest alternative slots if none work.

7. ****Meeting Details****

- Indicate the preferred meeting format (e.g., virtual via Zoom, in-person at your office, etc.).
- If virtual, mention that a meeting link will be provided upon confirmation.

8. ****Call to Action****

- Politely ask the client to confirm a convenient time or propose another.
- Express appreciation for their time and consideration.

9. ****Closing****

- Use a courteous closing (e.g., "Best regards," or "Sincerely,") followed by the signature block.

10. ****Tone and Style****

- Maintain a formal yet friendly tone.
- Keep the email concise (3-5 short paragraphs).
- Use proper business email etiquette (no slang, correct punctuation, and proper capitalization).

****Output Requirement:**** Provide only the finished email text, with all placeholders intact, ready for you to replace with actual details. Do not include any explanatory notes or additional

```
email_response = groq_client.chat.completions.create(
    model="openai/gpt-oss-120b",
    messages=[
        {"role": "user", "content": new_prompt}
    ]
)

print(email_response.choices[0].message.content)
```

Subject: Request for Meeting Next Week - [Topic/Project]

Dear [Client Name],

I hope you are well. Following our recent conversation about [Topic/Project] and the progress we have made at [Client Company], I would like to schedule a brief meeting next week to discuss

Would any of the following times work for you? (All times are in [Time Zone])

- [Day, Date] at [Time] - [Time Zone]
- [Day, Date] at [Time] - [Time Zone]
- [Day, Date] at [Time] - [Time Zone]

If none of these slots are convenient, please feel free to suggest alternative times that suit your schedule. I propose a virtual meeting via Zoom; a meeting link will be provided once we

Thank you for your consideration. I look forward to your reply.

Best regards,

[Your Name]
[Your Title]
[Your Company]
Phone: [Your Phone]
Email: [Your Email]

7) JSON Prompting

JSON Prompting is a technique where you instruct an AI model to return its output in **JSON (JavaScript Object Notation)** format — a structured, machine-readable way of representing data.

Instead of producing free-flowing text, the model outputs information in a structured schema (with keys and values).

Why JSON Prompting?

- Ensures **consistent structure** for downstream systems.
- Makes it easy to **parse and integrate outputs** into software, dashboards, or APIs.
- Useful in **automation, reporting, and data pipelines**.

Why it Works?

The effectiveness of JSON Prompting it leverages how LLMs actually learn. During training, these models encounter millions of examples of structured data: API responses, configuration files, database schemas, and code documentation. JSON patterns are deeply embedded in their understanding.

JSON Prompting transforms AI from an unpredictable tool into a reliable system component. Teams can build automated workflows knowing that outputs will consistently match expected formats, reducing the need for complex error handling and manual intervention.

Consider the following example:

Traditional Prompt:

"Analyze this customer review and tell me about the sentiment"

JSON Prompt:

```
{  
  "task": "sentiment_analysis",  
  "input": "The product exceeded my expectations!",  
}
```

```

"output_format": {
  "sentiment": "positive|negative|neutral",
  "confidence": "0.0-1.0",
  "key_phrases": ["array", "of", "strings"],
  "summary": "brief explanation"
}
}

```

```
# Json Prompt example
```

```

json_prompt = """{
  "task": "sentiment_analysis",
  "input": "The product exceeded my expectations!",
  "output_format": {
    "sentiment": "positive|negative|neutral",
    "confidence": "0.0-1.0",
    "key_phrases": ["array", "of", "strings"],
    "summary": "brief explanation"
  }
}"""

response = groq_client.chat.completions.create(
  model="openai/gpt-oss-120b",
  messages=[
    {"role": "user", "content": json_prompt}
  ]
)

```

```

output = response.choices[0].message.content
print(output)          # <-- the output looks json but it is still a string

```

```

{
  "sentiment": "positive",
  "confidence": 0.97,
  "key_phrases": [
    "exceeded my expectations"
  ],
  "summary": "The user is highly satisfied, indicating the product performed better than they anticipated."
}

```

```
type(output)
```

```
str
```

```
print('\n'.join(output.split('\n')[1:-1]))
```

```

"sentiment": "positive",
"confidence": 0.97,
"key_phrases": [
  "exceeded my expectations"
],
"summary": "The user is highly satisfied, indicating the product performed better than they anticipated."

```

```
# Convert the output string to Json (Python dictionary)

import json

try:
    parsed_output = json.loads(output)
except:
    parsed_output = json.loads('\n'.join(output.split('\n')[1:-1]))

type(parsed_output)

dict
```

```
parsed_output
```

```
{'sentiment': 'positive',
 'confidence': 0.97,
 'key_phrases': ['exceeded my expectations'],
 'summary': 'The user is highly satisfied, indicating the product performed better than they anticipated.'}
```

```
# Now you can access fields like a dictionary
print(parsed_output["sentiment"])
print(parsed_output["confidence"])
print(parsed_output["key_phrases"])
print(parsed_output["summary"])
```

```
positive
0.97
['exceeded my expectations']
The user is highly satisfied, indicating the product performed better than they anticipated.
```

8) Dynamic Prompt Generation Using Jinja Templates

Jinja (commonly Jinja2) is a templating engine for Python that allows you to create dynamic text or documents by combining static text with variables and logic.

In simple terms, Jinja is a tool that lets you insert data and logic into text templates to generate final output dynamically.

Basic Examples for Jinja Template

Example 1: Variable substitution

```
# Define a Jinja Template

from jinja2 import Template

template_str = """
Hello {{ name }}!

{{ message }}

Regards,
```

```
Support Team
"""
```

```
template = Template(template_str)
```

```
# Render the Template
output = template.render(name="Aman", message="Thanks for visiting")
print(output)
```

```
Hello Aman!
```

```
Thanks for visiting
```

```
Regards,
Support Team
```

```
# Render the Template for different inputs
output = template.render(name="Rashi", message="We appreciate your feedback")
print(output)
```

```
Hello Rashi!
```

```
We appreciate your feedback
```

```
Regards,
Support Team
```

Example 2: If-else condition

```
# Define a Jinja Template

from jinja2 import Template

template_str = """
Hello {{ name }}!

{% if sentiment == "positive" %}
We're glad to hear that you had a great experience!

{% elif sentiment == "negative" %}
We're sorry your experience didn't go as expected. Thanks for bringing this to our attention. We will look into it right away.

{% else %}
We appreciate your feedback.

{% endif %}

Sincerely,
Support Team
"""

template = Template(template_str)
```

```
# Render the Template
output = template.render(name="Joy", sentiment="positive")
print(output)
```

Hello Joy!

We're glad to hear that you had a great experience!

Sincerely,
Support Team

```
# Render the Template for different inputs
output = template.render(name="Rashi", sentiment="negative")
print(output)
```

Hello Rashi!

We're sorry your experience didn't go as expected. Thanks for bringing this to our attention. We will look into it right away.

Sincerely,
Support Team

Example 3: Jinja + LLM Call

```
# Define a Jinja Template

from jinja2 import Template

template_str = """
You are an expert {{ domain }} consultant.

Task:
{{ task }}

Context:
{{ context }}

Instructions:
{% if tone == "formal" %}
- Use professional language
{% else %}
- Use simple and friendly language
{% endif %}
- Keep the response under {{ max_words }} words
"""
```

```
# Render the Template

template = Template(template_str)

prompt = template.render(
    domain="HR",
    task="Suggest employee engagement activities",
    context="Remote teams are feeling disconnected",
    tone="formal",
    max_words=100
)

print(prompt)
```

You are an expert HR consultant.

Task:
Suggest employee engagement activities

Context:
Remote teams are feeling disconnected

Instructions:

- Use professional language
- Keep the response under 100 words

```
# Send Prompt to LLM

response = groq_client.chat.completions.create(
    model="openai/gpt-oss-20b",
    messages=[
        {"role": "user", "content": prompt}
    ]
)

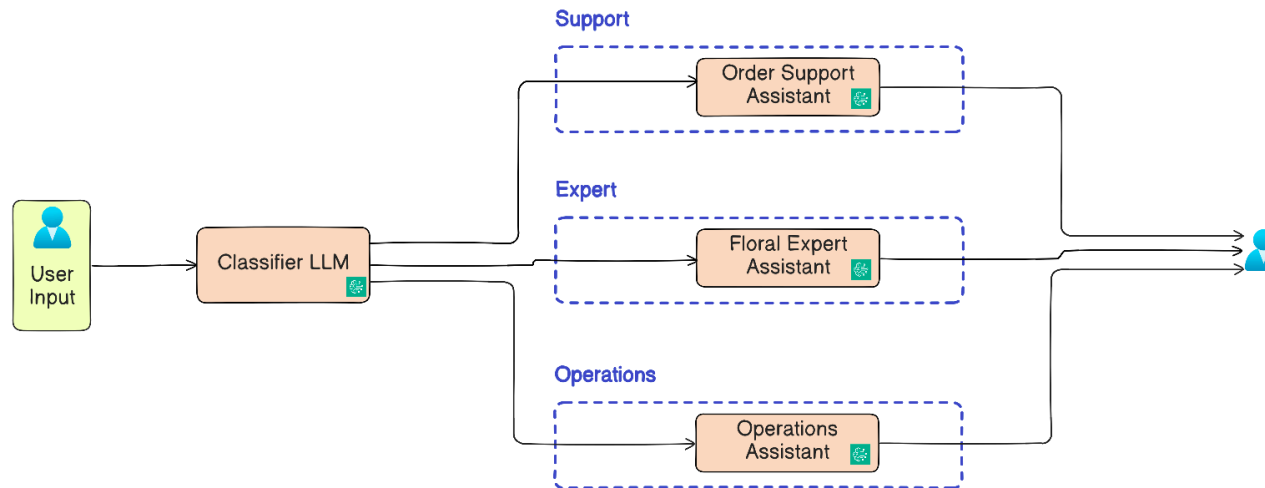
print(response.choices[0].message.content)
```

1. **Virtual coffee chats** - 15-minute informal video calls to spark casual conversation.
2. **Gamified team challenges** - virtual scavenger hunts or trivia that encourage friendly competition.
3. **Monthly recognition webinars** - peers spotlight each other's achievements in a structured format.
4. **Skill-sharing workshops** - employees present short tutorials on non-job topics to broaden collective knowledge.
5. **Wellness check-ins** - optional virtual yoga or mindfulness sessions for quick mental refreshers.
6. **Quarterly virtual retreats** - blend learning modules with social networking to strengthen team bonds.

ⅴ II. Practice Task

(Ungraded)

Create an **AI System** that can respond to customer and internal staff queries for a Flower Delivery Network.



Intermediate steps:

- **Create three different AI assistants**, each specializing in a different function of the flower delivery business:
 - **Order Support Assistant** – Handles customer issues like order status, delivery delays, bouquet customization, payment queries.
 - **Floral Expert Assistant** – Answers questions on flower types, care instructions, bouquet recommendations, seasonal availability.
 - **Logistics & Operations Assistant** – Handles driver routing, warehouse issues, delivery time estimates, packaging guidelines.

(Hint: Use Role-based System prompts. Try including Jinja templates.)

- **Route incoming user queries** to the correct assistant based on the nature of the question – Order Support / Floral Expert / Operations.

(Hint: Use Few-shot examples + JSON classification prompting to determine the category.)

- **Test the system** using sample customer and staff queries, for example:
 - *Can I change the delivery time for my bouquet?*
 - *Which flowers last longest in hot weather?*
 - *A driver is stuck in traffic—what should be the updated delivery estimate?*

YOUR CODE HERE...

YOUR CODE HERE...

YOUR CODE HERE...